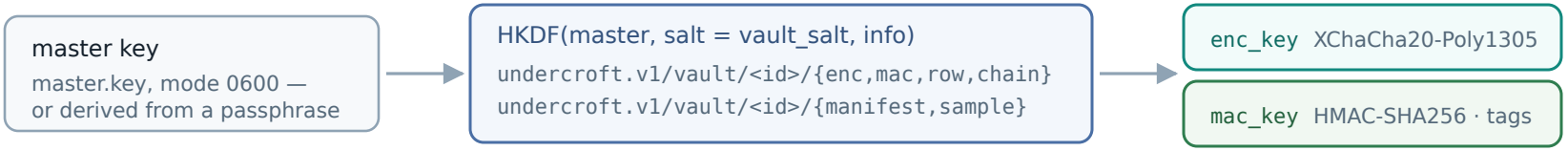
One master key, a separate key per vault, and an AAD that names the row

Isolation is cryptographic, not a WHERE clause: a blob lifted from one vault to another does not decrypt, because the vault id is authenticated.

Derivation



Six subkeys from one derivation: `enc` seals, `mac` tags records and fingerprints, `row` tags rows, `chain` steps the audit chain, `manifest` MACs the rollback anchor, `sample` draws the training rows — so a tag can never be forged from a decryption key.
All live in a zeroize-on-drop wrapper, are never Debug-printed, and the key file is fsynced at creation; a keycheck marker refuses a wrong passphrase up front rather than as a wall of decryption failures.
The graph's blind-index secret is NOT derived: 32 random bytes sealed in meta, re-sealed by rotation, never regenerated; else fact ids would move.

What every sealed blob authenticates alongside itself

The AEAD's associated data is not decoration — it is what makes a ciphertext meaningful only in the exact position it was written.

AAD = vault id + record id + domain

`<drawer id>`	content
`<drawer id>/emb`	embedding
`<drawer id>/tok`	ColBERT token matrix
`fde/<drawer id>/tok`	FDE row
`pqrow|pqpage|pq/…/pq`	PQ rows, pages, codebooks: not drawer ids
`kg/… · kgterms/…`	fact objects and terms · kgname/… names

so all of these FAIL, not merely return nothing

- A blob copied from vault A into vault B — the id differs.
- A row's ciphertext moved onto another row — the record id differs.
- An embedding swapped in where content was expected — the domain suffix differs.

Rotation — a new key, the same history

Every artifact is decrypted under the old key and re-sealed under the new one in a single transaction, and the chain is re-keyed over the audit bytes it already had — the history is preserved, not restarted.

The manifest is staged as `vault.json.next` and swapped, so a crash mid-rotation leaves one of two consistent states and is reconciled at the next open.

Sharing — an export bundle for one recipient

Hybrid X25519 + ML-KEM-768 to the recipient's public key, both shared secrets through HKDF into XChaCha20-Poly1305. The sender's vault keys never leave the machine, the bundle is readable by exactly one holder of the matching secret, and a hybrid recipient never silently downgrades to the curve half alone.

This is the no-lock-in path: a vault can be handed over whole without ever exposing it to the operator moving it.

Where the boundary honestly sits

Sealing protects data at rest. A running process holds the derived keys in memory, so an attacker with live access to the process, or an operator hosting the engine, is inside the boundary — no at-rest scheme changes that, and claiming otherwise would be dishonest.

What it does buy, precisely: a stolen disk, a leaked backup, a snapshot, a decommissioned drive, or a copied database file yields ciphertext and the metadata inventory — never the words.

The remote index tier is untrusted: a sealed vault pushes ciphertext plus embeddings and wing/room labels it reads in clear; hmac-only plaintext is refused without `--allow-plaintext`. Every push is chain-recorded; results are re-verified against the vault's HMAC before use.